Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada a Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquin

Ejercicios No. 6
Polimorfismo via Interfaces

Nombre: Raul Escobar
Carné: 26429
Sección # 30
Fecha de entrega: 03/11/2026

1. Análisis

1.1. Problema, objetivo y alcance

Una plataforma de comercio electrónico requiere concentrar el registro y la comunicación con APIs de pago, logística, rastreo y Factura Electrónica en Línea (FEL). Cada proveedor ofrece capacidades diferentes, por lo que el sistema no debe asumir que todos pueden cobrar, reintegrar, consultar o generar documentos.

Objetivo del diseño: ofrecer un catálogo único, extensible y ordenable de integraciones, con contratos por capacidad, interfaz de consola para administración y un motor de checkout que ignore de forma segura las operaciones incompatibles.

1.2. Requisitos extraídos del enunciado

ID	*Requisito verificable*	*Decisión de diseño*
RF-01	Registrar al iniciar ≥10 conexiones a APIs de naturaleza diversa.	InicializadorIntegraciones carga 10 objetos concretos en un único catálogo.
RF-02	Listar integraciones activas e informar estado de conexión.	Principal recorre el Iterable del catálogo y muestra activa/conectada.
RF-03	Buscar por identificador y revisar configuración.	Búsqueda por id y obtenerConfiguracion(), sin mostrar secretos.
RF-04	Ordenar catálogo por latencia promedio en ms.	Comparable<IntegracionAPI> y ordenamiento in situ ascendente.
RF-05	Admitir cobros, reintegros, consultas y documentos solo cuando la API los soporta.	Interfaces de capacidad Transaccional, Consultable y Documentable.
RF-06	Si se intenta reintegrar con logística, no detener el checkout ni lanzar excepciones.	MotorCheckout verifica contrato y devuelve ResultadoOperacion.IGNORADA.
RF-07	Agregar nuevas integraciones sin modificar el motor central.	Nuevas subclases usan los contratos existentes; se registran en el catálogo.
RNF-01	Usar herencia, polimorfismo, overloading, encapsulación y MVC.	IntegracionAPI abstracta, interfaces, sobrecarga y tres capas separadas.
RNF-02	Mantener una sola lista polimórfica para todos los ítems.	Solo CatalogoIntegraciones guarda List<IntegracionAPI>.

1.3. Actores, datos y reglas del negocio

Actor principal: administrador técnico del backend. Datos comunes por conexión: identificador único, nombre, endpoint, tipo de autenticación, timeout, latencia promedio, bandera activa y estado de conexión. Los identificadores son estables y no se editan; la configuración accesible al administrador no debe incluir claves ni tokens secretos.

- Cada objeto registrado debe ser un subtipo de IntegracionAPI; el identificador no se repite.
- Una integración activa puede estar desconectada: activa representa habilitación y conectada describe el estado de enlace.
- El catálogo conserva exactamente una lista; filtrar, buscar y ordenar no crean listas adicionales.
- No se ejecutan operaciones financieras sobre proveedores que no implementen Transaccional.
- Latencia ≥ 0 ms y timeout > 0 ms. Para datos inválidos se devuelve un resultado controlado, sin interrumpir el menú.

1.4. Identificación de componentes

Grupo	*Elementos*	*Responsabilidad*
Modelo base	IntegracionAPI, ResultadoOperacion, EstadoResultado	Estado común, identidad, comparación, resultado de operaciones.
Contratos	Auditable, Transaccional, Documentable, Consultable	Definir qué es capaz de hacer cada integración.
APIs concretas	PasarelaTarjeta, PasarelaTransferencia, BilleteraDigital, ProveedorLogistico, ServicioRastreo, CertificadorFEL	Representar proveedores de distinta naturaleza sin repetir atributos comunes.
Modelo / servicios	CatalogoIntegraciones, MotorCheckout	Guardar una lista polimórfica y despachar operaciones con seguridad.
Controlador	ControladorIntegraciones	Recibir peticiones del menú y coordinar las operaciones del modelo.
Vista y arranque	Principal, InicializadorIntegraciones	Consola y carga de datos de ejemplo, respectivamente.

1.5. Carga inicial propuesta: 10 integraciones

Los nombres y latencias siguientes son datos de prueba propuestos para el prototipo; no representan mediciones reales de proveedores. Se crean diez instancias, pero todas se guardan en la misma List<IntegracionAPI>.

Identificador	*Tipo de clase*	*Nombre ilustrativo*	*Latencia (ms)*	*Activa / conectada*
PAG-001	PasarelaTarjeta	Tarjeta Alfa	120	Sí / Sí
PAG-002	PasarelaTarjeta	Tarjeta Beta	95	Sí / Sí
PAG-003	PasarelaTransferencia	Transferencia Uno	180	Sí / Sí
PAG-004	BilleteraDigital	Billetera Río	75	Sí / Sí
PAG-005	BilleteraDigital	Billetera Sur	90	Sí / Sí
LOG-001	ProveedorLogistico	Envíos Express	245	Sí / Sí
LOG-002	ProveedorLogistico	Logística Central	210	Sí / No
CON-001	ServicioRastreo	Seguimiento Global	160	Sí / Sí
FEL-001	CertificadorFEL	FEL Uno	300	Sí / Sí
FEL-002	CertificadorFEL	FEL Dos	285	Sí / Sí

1.6. Casos de uso y respuestas esperadas

Caso	*Flujo / comprobación*
CU-01 Listar	Opción 1 → Controlador.listar() → el modelo entrega Iterable<IntegracionAPI> → Principal muestra solo activas y su estado (conectada/desconectada).
CU-02 Buscar	Opción 2 → ingresar id → Controlador.buscar(id) → la vista muestra obtenerConfiguracion() o "no encontrada".
CU-03 Ordenar	Opción 3 → ordenarPorLatencia() → Collections.sort() usa compareTo() → presentar latencias ascendentes y desempatar por id.
CU-04 Reintegrar	Opción 4 → ingresar id, monto y referencia → MotorCheckout.solicitarReintegro(). Si implementa Transaccional, ejecuta; si no, IGNORADA.
CU-05 Ampliar	Definir una nueva clase que extienda IntegracionAPI e implemente contratos apropiados; instanciarla y agregarla sin editar MotorCheckout.

2. Diseño

2.1. Arquitectura MVC y organización de responsabilidades

Capa	*Clases*	*Límite de responsabilidad*
Modelo	IntegracionAPI, subclases, contratos, ResultadoOperacion, EstadoResultado, CatalogoIntegraciones y MotorCheckout	Datos, reglas del dominio, ordenamiento, capacidades y lógica segura de negocio; sin impresión en consola.
Vista	Principal	Mostrar menú y resultados; leer entradas. Es la única clase que usa System.out.println y Scanner.
Controlador	ControladorIntegraciones	Intermediario entre menú y modelo. Recibe datos, busca, ordena y delega reintegros; devuelve objetos/resultados.
Inicialización	InicializadorIntegraciones	Instancia las 10 APIs de prueba una sola vez y las registra en el modelo; no contiene lógica de checkout.

El menú debe resolverse con una condición de salida (por ejemplo, opción 0) y sin usar while(true) seguido de break. Esto previene la penalización indicada para la futura implementación. La vista puede presentar también una opción de reintegro de prueba para demostrar la respuesta IGNORADA.

2.2. Contratos e interfaces

Interfaz	*Operación pública*	*Implementadores / criterio*
Auditable	registrarEvento(evento: String): String	IntegracionAPI la implementa una sola vez; cualquier subtipo hereda el contrato.
Transaccional	cobrar(monto: double, ref: String): ResultadoOperacion; reintegrar(monto: double, ref: String): ResultadoOperacion	PasarelaTarjeta, PasarelaTransferencia y BilleteraDigital.
Documentable	generarDocumento(ref: String): String	ProveedorLogistico (guía) y CertificadorFEL (XML de FEL).
Consultable	consultar(ref: String): String	ProveedorLogistico y ServicioRastreo (estado / seguimiento).
Comparable<IntegracionAPI>	compareTo(otra: IntegracionAPI): int	Implementada por IntegracionAPI para ordenar por latencia.

2.3. Única lista polimórfica y criterio de comparación

CatalogoIntegraciones declara exactamente un campo de colección: - integraciones: List<IntegracionAPI>, construido como new ArrayList<IntegracionAPI>(). Allí conviven las diez instancias de las seis subclases. Ningún otro componente posee un List o ArrayList para almacenar integraciones; listar devuelve una vista de lectura Iterable, buscar recorre la misma lista y ordenar actúa sobre ella in situ.

Implementación esperada de compareTo(): primero comparar latenciaPromedioMs en orden ascendente (Double.compare); si hay empate, comparar id de forma lexicográfica. equals() y hashCode() se basan en id; compareTo() resuelve empates por id para que el ordenamiento sea determinista. No es necesario crear copias ni arreglos auxiliares en la lógica diseñada.

2.4. Tablas de propiedades

Propiedades: IntegracionAPI (abstracta)

Propiedad / visibilidad	*Tipo*	*Función y restricciones*
- id {private, final}	String	Identificador único; getter público, sin setter.
- nombre {private}	String	Nombre visible; getter/setter.
- endpoint {private}	String	URL base; getter/setter. No almacena credenciales.
- tipoAutenticacion {private}	String	Método configurado (sin exponer tokens); getter/setter.
- timeoutMs {private}	int	Tiempo máximo de espera; valor positivo.
- latenciaPromedioMs {private}	double	Latencia de red; 0 o mayor.
- activa {private}	boolean	Indica si se muestra como integración habilitada.
- conectada {private}	boolean	Estado real o simulado de conexión.

Propiedades: CatalogoIntegraciones

Propiedad / visibilidad	*Tipo*	*Función y restricciones*
- integraciones {private, final}	List<IntegracionAPI>	ÚNICA colección polimórfica del programa, creada como ArrayList<IntegracionAPI>.

Propiedades: ControladorIntegraciones

Propiedad / visibilidad	*Tipo*	*Función y restricciones*
- catalogo {private, final}	CatalogoIntegraciones	Referencia al único catálogo existente.
- checkout {private, final}	MotorCheckout	Referencia al motor; no mantiene nuevas colecciones.

Propiedades: ResultadoOperacion

Propiedad / visibilidad	*Tipo*	*Función y restricciones*
- estado {private, final}	EstadoResultado	EXITOSA, IGNORADA o FALLIDA.

- mensaje {private, final}	String	Mensaje claro para la vista, sin imprimir desde el modelo.
- referencia {private, final}	String	Identificador de la operación que se procesó.

Las interfaces no declaran atributos de instancia. Las seis clases de proveedor reutilizan los atributos encapsulados de IntegracionAPI sin duplicarlos. MotorCheckout e InicializadorIntegraciones no requieren campos propios; Principal únicamente conserva el Scanner de consola.

2.5. Tablas de métodos

Firma tipada del constructor base (reutilizada por las seis clases concretas, mediante super): IntegracionAPI(String id, String nombre, String endpoint, String tipoAutenticacion, int timeoutMs, double latenciaPromedioMs, boolean activa, boolean conectada). Cada constructor concreto recibe esos mismos ocho parámetros; los métodos específicos se detallan por clase.

Métodos: IntegracionAPI (abstracta)

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ IntegracionAPI(id, nombre, endpoint, tipoAutenticacion, timeoutMs, latenciaPromedioMs, activa, conectada)	Constructor	Inicializar y validar los datos comunes.
+ getId(), getNombre(), getEndpoint(), getTipoAutenticacion()	String	Lectura encapsulada de las propiedades.
+ getTimeoutMs(), getLatenciaPromedioMs()	int / double	Lectura de timeout y latencia.
+ isActiva(), isConectada()	boolean	Lectura de ambos indicadores de estado.
+ setNombre(valor), setEndpoint(valor), setTipoAutenticacion(valor)	void	Actualizar datos de configuración visibles.
+ setTimeoutMs(valor), setLatenciaPromedioMs(valor)	void	Actualizar con validaciones de rangos.
+ setActiva(valor), setConectada(valor)	void	Actualizar banderas sin modificar id.
+ obtenerConfiguracion()	String	Devolver parámetros y estados sin secretos.
+ registrarEvento(evento: String)	String	Cumplir Auditable; devolver registro informativo.
+ compareTo(otra: IntegracionAPI)	int	Comparar latencia y luego id.
+ toString()	String	Presentación resumida de nombre, id y estado.
+ equals(obj: Object)	boolean	Comparar identidad por id.
+ hashCode()	int	Producir hash consistente con equals.

Métodos: Interfaces

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
Auditable.registrarEvento(evento: String)	String	Contrato de trazabilidad; sin listas de eventos.
Transaccional.cobrar(monto: double, ref: String)	ResultadoOperacion	Cobro en API de pago.
Transaccional.reintegrar(monto: double, ref: String)	ResultadoOperacion	Reintegro en API de pago.
Documentable.generarDocumento(ref: String)	String	XML FEL o guía de envío.
Consultable.consultar(ref: String)	String	Rastreo o consulta de estado.
Comparable.compareTo(otra: IntegracionAPI)	int	Orden natural de integraciones.

Métodos: PasarelaTarjeta

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ PasarelaTarjeta(datos de configuración)	Constructor	Invocar super(...) y crear un proveedor de tarjeta.
+ cobrar(monto: double, ref: String)	ResultadoOperacion	Implementar Transaccional para cobro con tarjeta.
+ reintegrar(monto: double, ref: String)	ResultadoOperacion	Implementar reintegro de pago con tarjeta.

Métodos: PasarelaTransferencia

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ PasarelaTransferencia(datos de configuración)	Constructor	Invocar super(...) y crear pasarela bancaria.
+ cobrar(monto: double, ref: String)	ResultadoOperacion	Implementar cobro/transferencia.
+ reintegrar(monto: double, ref: String)	ResultadoOperacion	Implementar reversión o reintegro compatible.

Métodos: BilleteraDigital

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ BilleteraDigital(datos de configuración)	Constructor	Invocar super(...) y crear billetera.
+ cobrar(monto: double, ref: String)	ResultadoOperacion	Implementar cargo a billetera.
+ reintegrar(monto: double, ref: String)	ResultadoOperacion	Implementar reintegro de billetera.

Métodos: ProveedorLogistico

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ ProveedorLogistico(datos de configuración)	Constructor	Crear API de logística heredando configuración común.
+ consultar(ref: String)	String	Obtener estado/cotización de envío.
+ generarDocumento(ref: String)	String	Crear guía de envío.

Métodos: ServicioRastreo

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ ServicioRastreo(datos de configuración)	Constructor	Crear API dedicada únicamente a consulta.
+ consultar(ref: String)	String	Consultar seguimiento de paquete.

Métodos: CertificadorFEL

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ CertificadorFEL(datos de configuración)	Constructor	Crear API de certificación electrónica.
+ generarDocumento(ref: String)	String	Generar representación XML certificada (simulada).

Métodos: CatalogoIntegraciones

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ CatalogoIntegraciones()	Constructor	Inicializa una sola ArrayList<IntegracionAPI>.
+ agregar(api: IntegracionAPI)	boolean	Registrar API si el id no está duplicado.
+ listar()	Iterable<IntegracionAPI>	Exponer lectura sin crear otro catálogo/lista.
+ buscarPorId(id: String)	IntegracionAPI	Recorrer y devolver coincidencia, o null.
+ ordenarPorLatencia()	void	Ordenar la lista en su lugar mediante Comparable.
+ cantidad()	int	Número de objetos registrados.

Métodos: MotorCheckout

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ solicitarReintegro(api: IntegracionAPI, monto: double, ref: String)	ResultadoOperacion	Entrada segura: valida y comprueba instanceof Transaccional.
+ solicitarReintegro(servicio: Transaccional, monto: double, ref:	ResultadoOperacion	Sobrecarga: delega por interfaz, con motivo opcional.

String, motivo: String)		
+ solicitarCobro(servicio: Transaccional, monto: double, ref: String)	ResultadoOperacion	Cobrar por contrato, no por clase concreta.
+ solicitarDocumento(servicio: Documentable, ref: String)	String	Generar documentos sin depender del proveedor.
+ solicitarConsulta(servicio: Consultable, ref: String)	String	Consulta por capacidad.

Métodos: ControladorIntegraciones

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ ControladorIntegraciones(catalogo, checkout)	Constructor	Inyectar el modelo y el motor.
+ listar()	Iterable<IntegracionAPI>	Delegar listado al catálogo.
+ buscar(id: String)	IntegracionAPI	Delegar búsqueda.
+ ordenarPorLatencia()	void	Delegar ordenamiento.
+ reintegrar(id: String, monto: double, ref: String)	ResultadoOperacion	Resolver id y llamar entrada segura del checkout.
+ registrar(api: IntegracionAPI)	boolean	Agregar nueva API por clase base.

Métodos: ResultadoOperacion y EstadoResultado

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
+ ResultadoOperacion(estado, mensaje, referencia)	Constructor	Crear respuesta explícita, sin excepciones por capacidades.
+ getEstado(), getMensaje(), getReferencia()	Enum / String	Leer resultado inmutable.
+ toString()	String	Resumen para la vista.
EstadoResultado: EXITOSA, IGNORADA, FALLIDA	Enum	Estados posibles del procesamiento.

Métodos: Principal e InicializadorIntegraciones

Firma UML (+ público; - privado)	*Retorno*	*Propósito*
Principal.main(args: String[]) {static}	void	Cargar catálogo, instanciar controlador y mostrar menú.
Principal.mostrarMenu() {private}	void	Imprimir opciones y leer elección con salida controlada.
Principal.mostrarIntegraciones() {private}	void	Recorrer Iterable y mostrar activas con conexión.
Principal.mostrarDetalle(api: IntegracionAPI) {private}	void	Mostrar configuración o mensaje si api es null.
InicializadorIntegraciones.cargar(catalogo: CatalogoIntegraciones) {static}	void	Instanciar 10 APIs y agregarlas al único catálogo.

2.6. Polimorfismo, sobrecarga y seguridad de operaciones

El motor central recibe parámetros del tipo de interfaz adecuado: Transaccional para cobros y reintegros, Documentable para generación de documentos y Consultable para consultas. Cada clase concreta decide cómo responde al contrato; se sustituye un proveedor por otro sin cambios en el algoritmo central (polimorfismo vía interfaces).

Sobrecarga (overloading) intencional: MotorCheckout.solicitarReintegro(IntegracionAPI, double, String) funciona como entrada genérica protegida; MotorCheckout.solicitarReintegro(Transaccional, double, String, String) usa el contrato especializado y un motivo adicional. La cantidad diferente de parámetros evita ambigüedades cuando una clase concreta es a la vez IntegracionAPI y Transaccional.

Situación	*Regla y respuesta de diseño*
API de tarjeta + reintegro	La verificación detecta Transaccional; llama a la sobrecarga especializada y obtiene ResultadoOperacion (EXITOSA o FALLIDA).
API logística + reintegro	ProveedorLogistico NO implementa Transaccional. El método genérico devuelve ResultadoOperacion(IGNORADA, mensaje, referencia), sin cast inválido ni excepción.
Id no existente	Controlador busca y devuelve ResultadoOperacion FALLIDA

	con mensaje de integración inexistente.
Monto no positivo	El motor devuelve resultado FALLIDA y no intenta contactar la integración.
API desconectada	Se devuelve FALLIDA o se evita el envío; el mensaje permite al administrador identificar el estado.

2.7. Extensibilidad y consistencia del diseño

Ejemplo: la startup de última milla se modelaría como una nueva clase UltimaMilla que extiende IntegracionAPI e implementa Consultable y Documentable según sus servicios. Solo es necesario construir la instancia y registrarla mediante agregar(api); MotorCheckout no cambia porque solo conoce contratos. Esta nueva clase es una posibilidad de expansión, no forma parte de las 10 instancias iniciales.

Consistencia de interfaz: las integraciones que no generan documentos no implementan Documentable; las que no mueven dinero no implementan Transaccional. Esta separación respeta el principio de segregación de interfaces y evita implementar métodos artificiales que lancen excepciones.

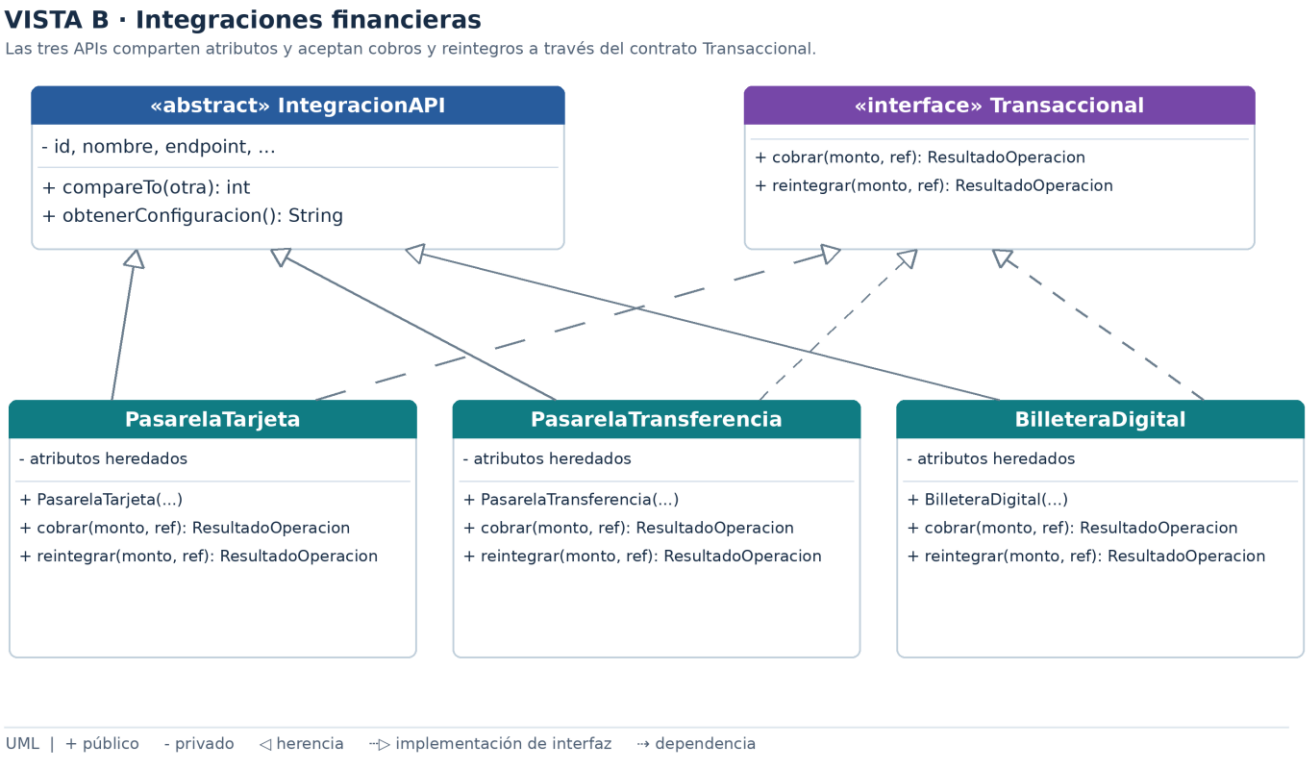
2.8. UML de clases: estructura base



Los símbolos + y - representan visibilidad. En los diagramas se sintetizan firmas largas; las tablas anteriores contienen las firmas completas.

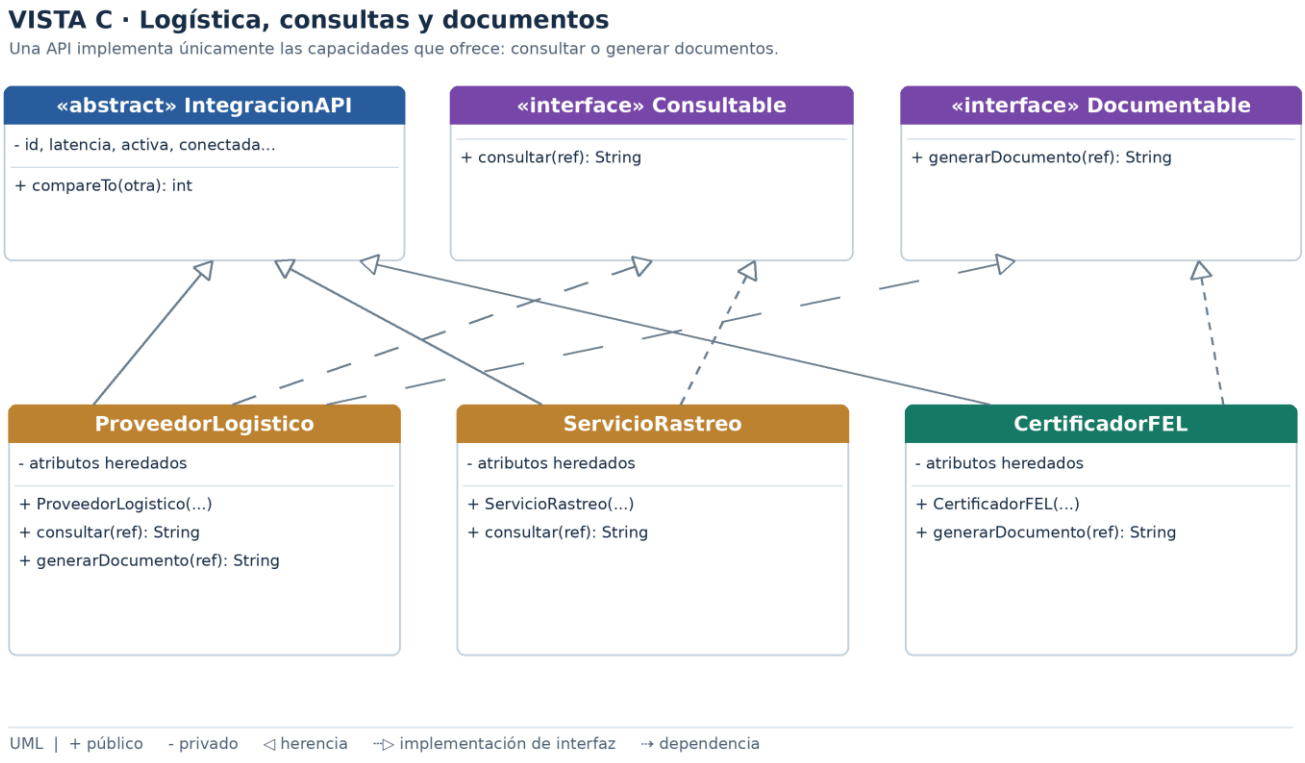
2.9. UML de clases: pasarelas de pago

Triángulos con línea continua: herencia. Triángulos con línea discontinua: realización de la interfaz Transaccional.



2.10. UML de clases: logística y FEL

Las clases implementan únicamente los contratos compatibles. ProveedorLogistico y ServicioRastreo no implementan Transaccional.



2.11. UML de clases: arquitectura MVC

Las dependencias expresan colaboración sin crear colecciones adicionales; todos los objetos están en el único catálogo polimórfico.

